

INFORME DE ESTADO Y ARQUITECTURA

PROYECTO PEDIDOS360

Desarrollo Cloud Native — Backend Microservicios, Persistencia JPA, Amazon RDS y AWS API Gateway

ASIGNATURA / EVALUACIÓN

Desarrollo Cloud Native I — Parcial 1

INFRAESTRUCTURA CLOUD

Amazon Web Services (AWS) — us-east-1

STACK TECNOLÓGICO

Java 21, Spring Boot 3.3.4, Angular 19, PostgreSQL 16

ESTADO DEL PROYECTO

Backend Operativo + Persistencia JPA + Planificación Cloud

3 / 3

MICROSERVICIOS OPERATIVOS

80 / 80

TESTS AUTOMATIZADOS OK

100%AISLAMIENTO DE RED Y
SEGURIDAD

1. Resumen Ejecutivo

El proyecto **Pedidos360** es una plataforma web cloud native diseñada bajo una arquitectura desacoplada de microservicios, orientada a la alta disponibilidad, seguridad y mantenibilidad en la nube de **Amazon Web Services (AWS)**.

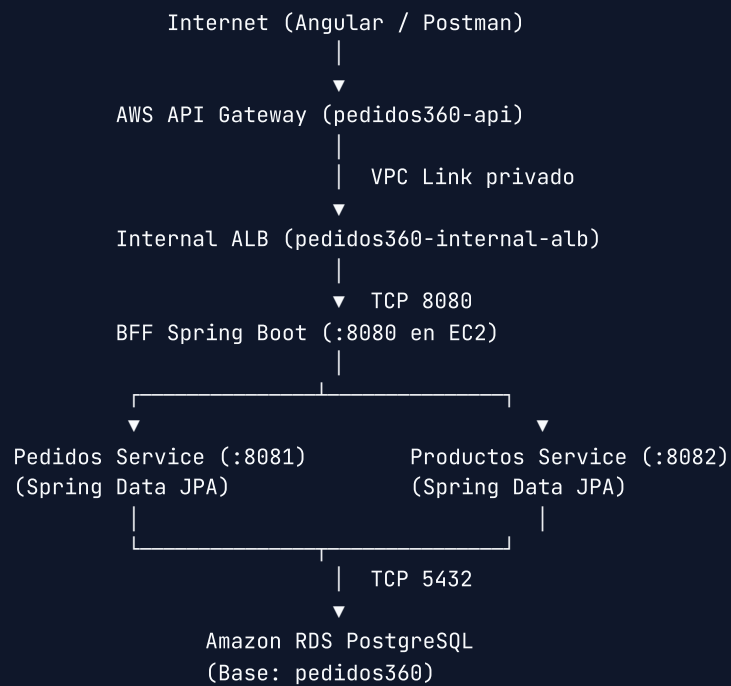
A la fecha, el proyecto cuenta con su infraestructura base en AWS operativa en la región `us-east-1`, con despliegue verificado en Amazon EC2 mediante servicios systemd, persistencia relacional implementada mediante Spring Data JPA para PostgreSQL, y planes exhaustivos de integración para Amazon RDS y AWS API Gateway.

Hitos Clave Completados:

- **EC2 Desplegado y Activo:** Microservicios ejecutándose con Amazon Corretto 21 bajo systemd (BFF en `0.0.0.0:8080`, Pedidos en `127.0.0.1:8081` y Productos en `127.0.0.1:8082`).
- **Persistencia JPA Completa:** CRUDs de Productos y Pedidos con entidades, DTOs, validaciones con Jakarta Validation y manejo global de excepciones en JSON.
- **Aislamiento y Seguridad:** Cero secretos en repositorios Git, comunicación inter-servicios restringida a loopback local y pruebas automatizadas 100% aisladas con H2.
- **Control de Versiones:** Repositorio Git limpio, estructurado y versionado con commits descriptivos bajo estándar Conventional Commits.

2. Arquitectura del Sistema

La arquitectura global sigue el patrón **Backend For Frontend (BFF)**, donde el cliente web no consume directamente los microservicios de dominio ni la base de datos, sino que interactúa con un API Gateway público que enruta el tráfico hacia el BFF.



3. Microservicios Backend y Persistencia JPA

3.1 BFF Service (bff-service)

- **Puerto:** 8080 (escucha en 0.0.0.0:8080).
- **Rol:** Puerta de enlace interna hacia los microservicios de dominio. Orquesta peticiones y centralizará la validación de tokens JWT en etapas posteriores.
- **Aislamiento de BD:** No contiene configuración ni dependencias directas de JPA/PostgreSQL.
- **Health Checks:** GET /api/health y GET /actuator/health respondiendo HTTP 200 UP.

3.2 Productos Service (productos-service)

- **Puerto:** 8082 (escucha en 127.0.0.1:8082 , no expuesto públicamente).
- **Entidad Producto :** Campos id (Long), nombre (String), descripcion (String), precio (BigDecimal), stock (Integer), activo (Boolean), createdAt y updatedAt .
- **Operaciones CRUD implementadas:**

Método	Endpoint	Respuesta Exitosa	Descripción
POST	/api/productos	201 CREATED	Crea un nuevo producto validado.
GET	/api/productos	200 OK	Lista productos (activos por defecto o todos).
GET	/api/productos/{id}	200 OK	Busca un producto por su ID (404 si no existe).
PUT	/api/productos/{id}	200 OK	Actualiza datos de un producto existente.
DELETE	/api/productos/{id}	204 NO CONTENT	Eliminación lógica del producto (activo=false).

3.3 Pedidos Service (pedidos-service)

- **Puerto:** 8081 (escucha en 127.0.0.1:8081 , no expuesto públicamente).
- **Entidad Pedido :** Campos id (Long), usuario (String), fecha (LocalDateTime), estado (Enum EstadoPedido : PENDIENTE, CONFIRMADO, EN_PROCESO, COMPLETADO, CANCELADO persistido como STRING), total (BigDecimal), createdAt y updatedAt .
- **Operaciones CRUD implementadas:**

Método	Endpoint	Respuesta Exitosa	Descripción
POST	/api/pedidos	201 CREATED	Registra un nuevo pedido.
GET	/api/pedidos	200 OK	Lista todos los pedidos registrados.
GET	/api/pedidos/{id}	200 OK	Busca un pedido por ID (404 si no existe).
GET	/api/pedidos/usuario/{usuario}	200 OK	Filtra pedidos por usuario/cliente.
PUT	/api/pedidos/{id}	200 OK	Actualiza estado o monto del pedido.
DELETE	/api/pedidos/{id}	204 NO CONTENT	Elimina el registro de pedido.

4. Infraestructura en Amazon Web Services (AWS)

4.1 Instancia Amazon EC2

Parámetro	Valor	Notas
Instancia ID	i-003c43af8dfb9a3c5	Instancia dedicada para backend Pedidos360.
Tipo de Instancia	t3.medium	2 vCPU, 4 GiB RAM, arquitectura x86_64.
Sistema Operativo	Amazon Linux 2023	JDK 21 Amazon Corretto, systemd activado.
IAM Role	LabRole / Pedidos360Ec2InstanceProfile	Soporte nativo para AWS Systems Manager (SSM).
Mecanismo de Despliegue	S3 Privado + SSM Run Command	Despliegue automatizado sin llaves SSH ni SCP.

4.2 Base de Datos Amazon RDS PostgreSQL

Propiedad	Configuración	Propósito
Motor / Versión	PostgreSQL 16.3	Compatibilidad con Spring Boot 3.3.4 y Hibernate 6.5.
Identificador BD	pedidos360-db	Base de datos inicial: pedidos360 .
Clase de Instancia	db.t3.micro	Optimizado para laboratorios de bajo costo.
Acceso Público	FALSE (PRIVADO)	Ubicado exclusivamente dentro de subredes de la VPC.
Cifrado	HABILITADO	Cifrado en reposo con AWS KMS default.
Security Group	pedidos360-rds-sg	Inbound TCP 5432 exclusivo desde sg-000542deedef64065 .

4.3 Matriz de Security Groups y Reglas de Red

Security Group	Puerto	Protocolo	Origen Autorizado	Finalidad
pedidos360-backend-sg	22	TCP	IP_ADMIN/32	Administración temporal (preferir SSM).
pedidos360-backend-sg	8080	TCP	pedidos360-alb-sg	Tráfico entrante exclusivo desde el balanceador.
pedidos360-rds-sg	5432	TCP	pedidos360-backend-sg	Conexión exclusiva microservicios → base de datos.
pedidos360-alb-sg	80	TCP	pedidos360-vpclink-sg	Tráfico HTTP interno desde el VPC Link.
pedidos360-vpclink-sg	Egress	TCP	VPC CIDR (172.31.0.0/16)	Comunicación hacia subredes del balanceador.

5. Pruebas Automatizadas y Calidad de Código

Se implementaron suites exhaustivas de pruebas unitarias y de integración para cada microservicio. Las pruebas no dependen de servicios cloud activos ni bases de datos remotas, utilizando H2 en memoria bajo perfil de pruebas.

Módulo	Tests Ejecutados	Fallos	Errores	Cobertura Principal
bff-service	12	0	0	Health endpoints, perfiles Spring, configuración.
pedidos-service	30	0	0	CRUD Pedido, validaciones Jakarta, repositorios JPA, perfiles.
productos-service	28	0	0	CRUD Producto, eliminación lógica, validaciones, repositorios JPA.
TOTAL	80	0	0	100% Tests Pasados exitosamente.

5.1 Hashes de Integridad de Artefactos (SHA256)

```
bff-service.jar:
SHA256: BEB6FBBBCBD19E5197600790F6A8186BBDCE4F2D81864A1BC4C4ADFA75E98033F

pedidos-service.jar:
SHA256: 8901E9C74A7CB35E492969CEE2FD1480C2E3B8CE81FFB0CC600649670DB08706

productos-service.jar:
SHA256: 53D2ADC0B3B58A4A7927BDB4BCF5935C031225BFCA12F7902857D8E29898E0EE
```

6. Próximos Pasos (Hoja de Ruta Final)

- Microsoft Entra ID (Azure AD):** Registro de aplicación frontend (Single Page App) y backend API con scopes y roles.
- Angular + MSAL:** Integración de `@azure/msal-angular` y `@azure/msal-browser` con `MsalGuard` e interceptor Bearer Token.
- Seguridad JWT en BFF:** Configuración de Spring Security OAuth2 Resource Server para validación de firma, issuer y audiencia con JWKS de Microsoft.
- AWS API Gateway:** Aprovisionamiento de HTTP API `pedidos360-api` con enrutamiento proxy y CORS para `http://localhost:4200`.
- Frontend Hosting en AWS:** Despliegue de Angular en Amazon S3 + Amazon CloudFront con CDN y HTTPS.

6. **Presentación y Demo en Vivo:** Demostración de flujos 200 / 201 / 204 / 400 / 401 / 403 / 404 y persistencia en RDS.

Conclusión: El proyecto se encuentra en un estado óptimo, con arquitectura desacoplada, código limpio, pruebas pasando en su totalidad y listo para la fase de autenticación federada y hosting web.